

Bérlők CRUD műveletei

Bérlők módosítása:

A bérlő módosítása funkció elkészítése során lehetővé tettem a bérlők adatainak frissítését az adatbázisban. A rendszer képes a kiválasztott bérlő személyes adatainak módosítására és mentésére. A fejlesztés során létrehoztam az adatbázis frissítéséhez szükséges SQL lekérdezéseket. A módosítás sikeres végrehajtása esetén a rendszer visszajelzést küld a felhasználó számára. A funkció helyes működését Postman tesztekkel ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    nev,
    email,
    telefonszam,
    személyiszam,
    lakcim
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE users
      SET
        nev = ?,
        email = ?,
        telefonszam = ?,
        személyiszam = ?,
        lakcim = ?
      WHERE id = ?`,
      [
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim,
        req.params.id
      ]
    );
  }
});
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
        ]
    );

    res.json({
        siker: true,
        uzenet: "Bérlő módosítva!"
    });

} catch (err) {

    console.error(err);

    res.status(500).json({
        siker: false,
        uzenet: "Hiba történt!"
    });

} finally {

    if (conn) {
        conn.release();
    }

}

});

module.exports = router;
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

The screenshot shows the Postman interface for a PUT request to `http://localhost:3001/api/berlomodositas/1`. The request body is a JSON object: `{ "nev": "Kiss Bela Modositott", "email": "modositott@gmail.com", }`. The response is a 200 OK status with a response time of 8 ms and a body size of 309 B. The response body is a JSON object: `{ "siker": true, "uzenet": "Berlo modositva!" }`.

1. Részlet az `berlomodositas.js` fájlból.

Bérlők törlése:

A bérlő törlése funkció fejlesztése során elkészítettem a kiválasztott bérlő rekordjának eltávolítására szolgáló API végpontot. A rendszer az azonosító alapján megkeresi és törli a megfelelő adatot az adatbázisból. A fejlesztés során figyelmet fordítottam a biztonságos adatkezelésre és a hibakezelésre is. Sikeres törlés esetén a végpont tájékoztató üzenetet küld vissza a kliensnek. A funkció működését Postman alkalmazás segítségével teszteltem és ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      "DELETE FROM users WHERE id = ?",
      [req.params.id]
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
    );

    res.json({
      siker: true,
      uzenet: "Bérlő törölve!"
    });

  } catch (err) {

    console.error(err);

    res.status(500).json({
      siker: false,
      uzenet: "Hiba történt!"
    });

  } finally {

    if (conn) {
      conn.release();
    }

  }

});

module.exports = router;
```

HTTP http://localhost:3001/api/berlotories/1 Save

DELETE http://localhost:3001/api/berlotories/1 **Send**

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

This request does not have a body

Body Cookies Headers (8) Test Results 200 OK 10 ms 307 B **Save Response**

Pretty Raw Preview Visualize JSON

```
1  {
2    "siker": true,
3    "uzenet": "Berlo torolve!"
4  }
```

Bérlők listázás:

A bérlők listázása funkció fejlesztése során elkészítettem az összes bérlő adatainak lekérdezésére szolgáló API végpontot. A rendszer az adatbázisból lekéri a bérlők adatait és JSON formátumban továbbítja azokat a frontend számára. A fejlesztés célja az volt, hogy a felhasználók egyszerűen áttekinthető listában jeleníthessék meg a bérlőket. A lekérdezés során biztosítottam az adatbázis-kapcsolat megfelelő kezelését. Az elkészült végpont működését Postman segítségével sikeresen teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      `SELECT
        id,
        nev,
        email,
        telefonszam,
        szerekor,
        létrehozasdatum
      FROM users
      WHERE szerekor = 'berlo'`
    );

    res.json(adatok);

  } catch (err) {

    console.error(err);

    res.status(500).json({
      siker: false,
      uzenet: "Hiba történt a bérlők lekérdezése során!",
      hiba: err.message
    });

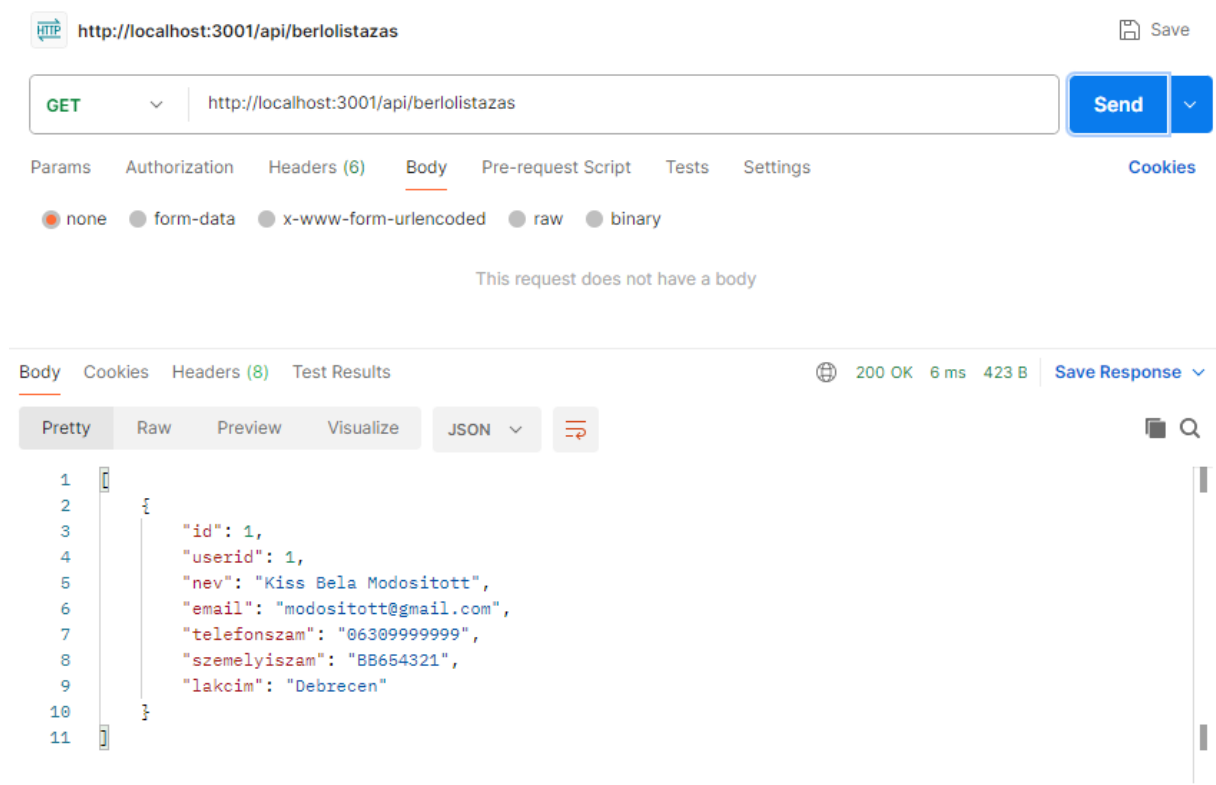
  } finally {
```

```
        if (conn) {
            conn.release();
        }

    }

});

module.exports = router;
```



4. Részlet az `berlolistazas.js` fájlból.

Bérlő hozzárendelése ingatlanhoz:

A bérlő hozzárendelése funkció fejlesztése során elkészítettem azt az API végpontot, amely lehetővé teszi egy bérlő és egy ingatlan összekapcsolását a rendszerben. A megvalósítás során a hozzárendelés egy új bérleti szerződés létrehozásával történt, amely tartalmazza az ingatlan és a bérlő azonosítóját, valamint a szerződéshez kapcsolódó adatokat. A rendszer eltárolja a szerződés kezdeti és végdátumát, a havi bérleti díjat és a szerződés aktuális állapotát is. A fejlesztés során biztosítottam, hogy a hozzárendelés csak létező bérlő és ingatlan rekordok között történhessen meg az adatbázis idegen kulcs kapcsolatain keresztül. Az elkészült funkció segítségével nyilvántarthatóvá vált, hogy melyik bérlő melyik ingatlanhoz tartozik. A végpont működését Postman alkalmazás segítségével teszteltem, ahol ellenőriztem az adatok sikeres rögzítését és az adatbázisban történő megjelenését.

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    ingatlanid,
    berloid,
    kezdetdatum,
    vegdatum,
    havidij,
    statusz
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO szerzodesek
      (ingatlanid, berloid, kezdetdatum, vegdatum, havidij, statusz)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [
        ingatlanid,
        berloid,
        kezdetdatum,
        vegdatum,
        havidij,
        statusz
      ]
    );

    res.json({
      siker: true,
      uzenet: "Bérlő sikeresen hozzárendelve az ingatlanhoz!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
    }  
  
  });  
  
module.exports = router;
```

The screenshot displays a REST client interface for a POST request to `http://localhost:3001/api/berlohozzarendeles`. The request body is a JSON object: `{\"ingatlanid\": 2, \"berloid\": 1}`. The response is a 200 OK status with a response time of 8 ms and a body of `{\"siker\": true, \"uzenet\": \"Bérlő sikeresen hozzárendelve az ingatlanhoz!\"}`.

5. Részlet az `berlohozzarendeles.js` fájlból.